

ESP32 HTTP Endpoint – Production Checklist (Extended Edition)

A practical validation and hardening guide for developers using the ESP32_IDF_HTTP_ENDPOINT Lite repository as a foundation for production-ready firmware.

Version: 1.0

1. Purpose of This Document

This checklist is designed to help embedded developers transition an ESP-IDF HTTP endpoint project from a functional demo (Lite) to a stable and production-ready implementation.

It focuses on validation, structure, reliability, security, and maintainability.

2. Network Initialization Validation

- Wi-Fi credentials configurable via menuconfig or secure storage.
- System waits for IP acquisition before starting HTTP server.
- Reconnect logic implemented on Wi-Fi disconnection.
- Proper event handler registration and cleanup.
- IP address logged clearly at boot.

3. HTTP Server Initialization

- `httpd_start()` return value checked.
- Server configuration reviewed (stack size, task priority, max URI handlers).
- Server starts only after network readiness.
- Graceful shutdown logic available (`httpd_stop`).
- Timeout values reviewed for expected use case.

4. Endpoint Architecture Review

- Clear separation between HTTP layer and business logic.
- Each URI handler performs minimal logic and delegates processing.
- Query parameters validated before usage.
- JSON responses structured consistently.
- Error codes returned appropriately (400, 404, 500).

5. Memory and Resource Management

- Heap usage monitored before and after HTTP requests.
- No dynamic allocation without proper `free()`.
- Stack size validated under stress conditions.
- Large buffers avoided or statically allocated when possible.
- No memory leaks after repeated endpoint calls.

6. Error Handling and Input Validation

- Missing parameters handled gracefully.
- Invalid values return clear error messages.
- Boundary conditions tested (max/min values).
- Malformed HTTP requests tested.
- Unexpected states logged with warning level.

7. Logging Strategy

- Log levels configurable (INFO, WARN, ERROR).
- Sensitive data not printed in logs.
- HTTP request entry logged minimally.
- Production build reduces verbose debug output.

8. Basic Security Hardening

- Input sanitization applied to query strings.
- Optional authentication mechanism considered (token/header-based).
- Server bound only to required interface.
- Rate limiting strategy evaluated.
- HTTPS feasibility evaluated if required by product scope.

9. Functional Testing Scenarios

Test using curl or Postman:

```
curl "http://<ESP_IP>/api/led?state=on"
```

```
curl "http://<ESP_IP>/api/led?state=off"
```

```
curl "http://<ESP_IP>/api/led?level=1"
```

- Correct JSON response returned.
- GPIO/LED state matches request.
- Multiple sequential requests do not crash system.
- Invalid request does not reboot device.

10. Pre-Release Validation Checklist

- Firmware built with production flags.
- Watchdog enabled and tested.
- Power-cycle tested multiple times.
- Stress test performed (100+ requests).
- Memory usage recorded and documented.
- Version number incremented and documented.

11. Upgrade Path to Professional Template

If you are planning to scale this Lite HTTP example into a reusable product component, consider implementing:

- Modular endpoint registration system.
- Middleware layer abstraction.
- Authentication and role management.
- Structured JSON response framework.
- Unit testable handler modules.
- Integration with OTA system.

12. Final Notes

This extended checklist is intended to help developers avoid common pitfalls when exposing HTTP APIs on ESP32 devices. While the Lite repository provides a clean starting point, production firmware requires structured validation and disciplined architecture.

Use this document as a repeatable release validation template for all future HTTP-based projects.